

Practical Assignment
CC2008- Machine Learning I (2024/2025)

PL5_G7
Ana Duarte Amorim - up202207213
Mariana de Sousa Serralheiro - up202208926

SUMÁRIO

1. K-Nearest Neighbor e o Problema do Noise e Outliers
 - a. Comportamento do Algoritmo perante os problemas de Noise e Outliers
2. Proposal
 - a. Motivação
 - b. Descrição
3. Estudo Empírico
 - a. Impacto de Diferentes Níveis de Outliers
 - b. Impacto de Diferentes Níveis de Noise
 - c. Hiperparâmetro K
 - d. Estimativas de Desempenho
4. Análise Estatística dos Resultados
5. Análise dos Resultados Face à Hipótese Inicial
6. Conclusão e Trabalhos futuros
7. Bibliografia
8. Anexos

K-NEAREST NEIGHBOR E O PROBLEMA DO NOISE E OUTLIERS

Descrição

O algoritmo que escolhemos para este estudo foi o **K-Nearest Neighbors (KNN)**.

O **KNN** é um algoritmo de classificação de **Machine Learning** que classifica um novo exemplo, com base nas classes dos seus *k vizinhos mais próximos*. Para determinar a distância entre os exemplos, utilizamos a **distância Euclidiana**. Definimos o número de vizinhos a analisar (*k*) como 5.

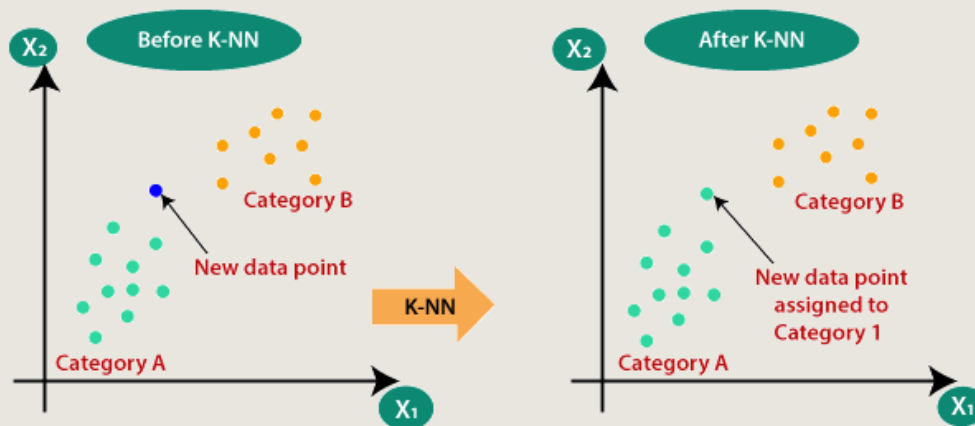
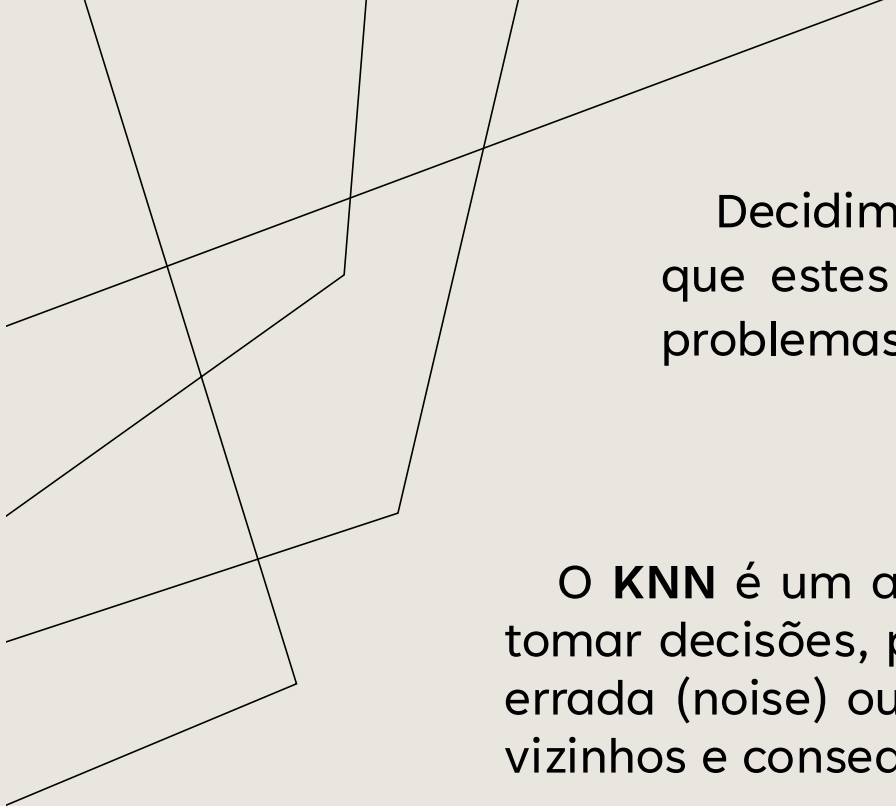


Fig.1 Algoritmo K-NN



Decidimos estudar o impacto do **Noise** e **Outliers** porque consideramos que estes problemas têm um pior impacto no algoritmo KNN do que os problemas de **Class Imbalance** ou **Multiclass Classification**.

O **KNN** é um algoritmo que depende muito da distância entre os exemplos para tomar decisões, por isso qualquer alteração, seja através de um dado com a classe errada (noise) ou com um outlier, pode afetar de forma significativa a escolha dos vizinhos e consequentemente a classificação final.

O problema de **Class Imbalance** também afeta este algoritmo mas não de forma tão imediata, porque o algoritmo consegue funcionar de forma razoável se os vizinhos mais próximos dos exemplos de teste tiverem as classes minoritárias.

Por outro o lado, o KNN adapta-se ao problema da **Multiclass Classification**.

Comportamento do Algoritmo perante os problemas de Noise e Outliers

Para estudar o **impacto do Noise e Outliers** no algoritmo **K-Nearest Neighbor** criamos funções que introduzissem estes problemas aos datasets dados.

Assim, quando damos o dataset escolhido ao algoritmo, ele questiona, sobre os **níveis de noise e outliers** que queremos *introduzir nos datasets*, e posteriormente, retorna os resultados das métricas de avaliação para esses datasets.

```
Insira o path do dataset: /home/ana/Desktop/AC I/Projeto/noise_outliers/dataset_451_irish.csv
Insira o nível de noise (entre 0 e 1): 0.3
Insira o nível de outliers (entre 0 e 1): 0.3
```

Para comparar os resultados dos valores das métricas de classificação entre o dataset original e os datasets com a introdução de noise e ruído. Optamos por escolher **níveis de noise e ruído de 30%**.

No geral, verificamos que os datasets originais apresentam melhores valores nas métricas de avaliação, como podemos observar pelo gráfico de comparação das métricas do dataset **451_Irish**. Este resultado confirma que o K-Nearest Neighbor, é muito sensível a noise e outliers.

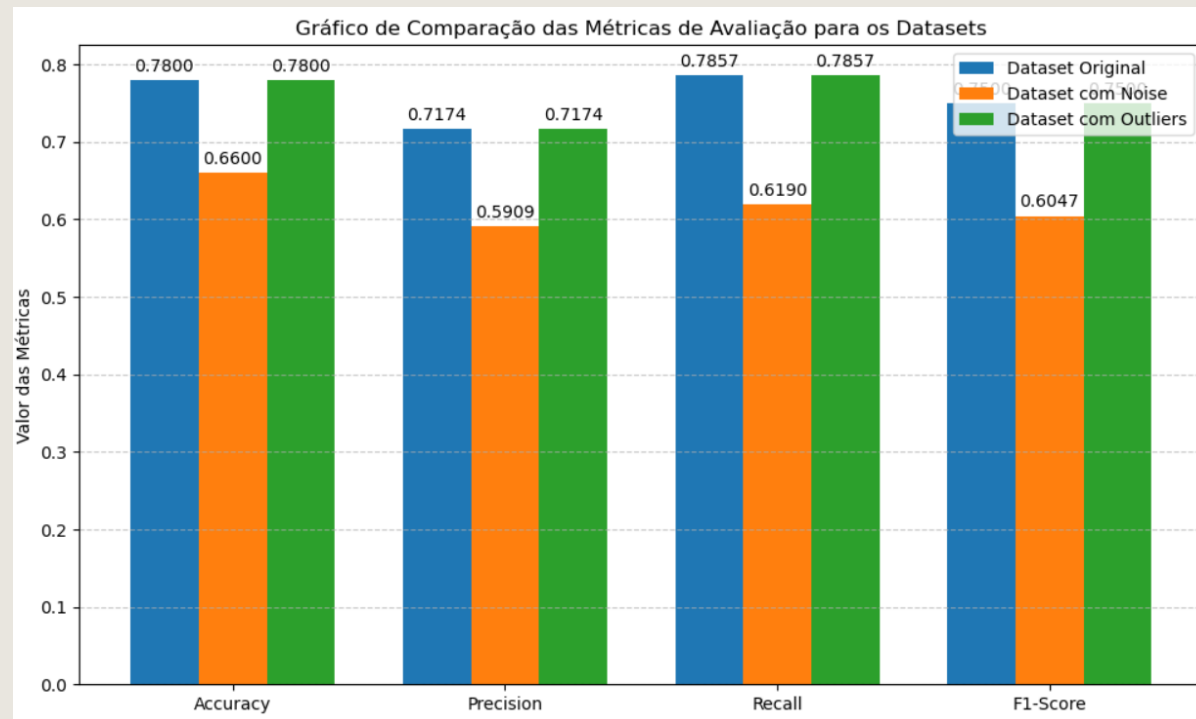


Fig.2 Gráfico de Comparação de Métricas de Avaliação para o Dataset 451_Irish

Curiosamente, em alguns casos específicos como é o caso do dataset **50_tic-tac-toe**, os datasets **com outliers** obtiveram métricas **ligeiramente superiores** ao dataset original.

Isto poderá ser explicado por:

- alguns outliers, como são criados aleatoriamente, acabaram por **reforçar a separação** entre as classes.
- se o dataset já continha problemas de noise ou outliers, adicionar mais dados fez com que os erros antigos tivessem menos peso

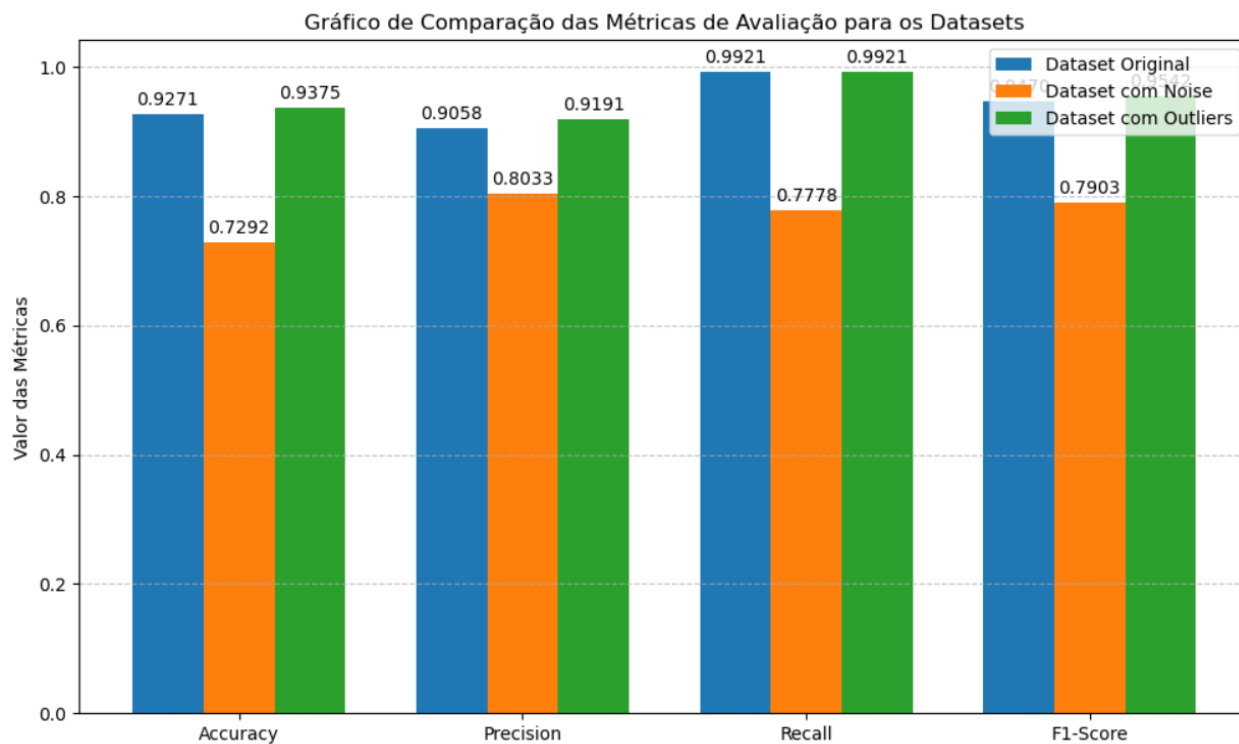


Fig.3 Gráfico de Comparação de Métricas de Avaliação para o Dataset 50_tic-tac-toe

Também observamos que em alguns datasets, a introdução de uma pequena quantidade de **noise** **melhorou ligeiramente o desempenho** do modelo, como podemos observar pelo gráfico do dataset **738_pharynx**.

Pequenas quantidades de noise atuaram como uma forma de **regularização espontânea**, uma vez que quebraram padrões artificiais nos dados de treino.

Isto forçou o algoritmo a não depender de relações demasiado específicas, tornando-o **menos propenso a overfitting** e **mais capaz de generalizar**.

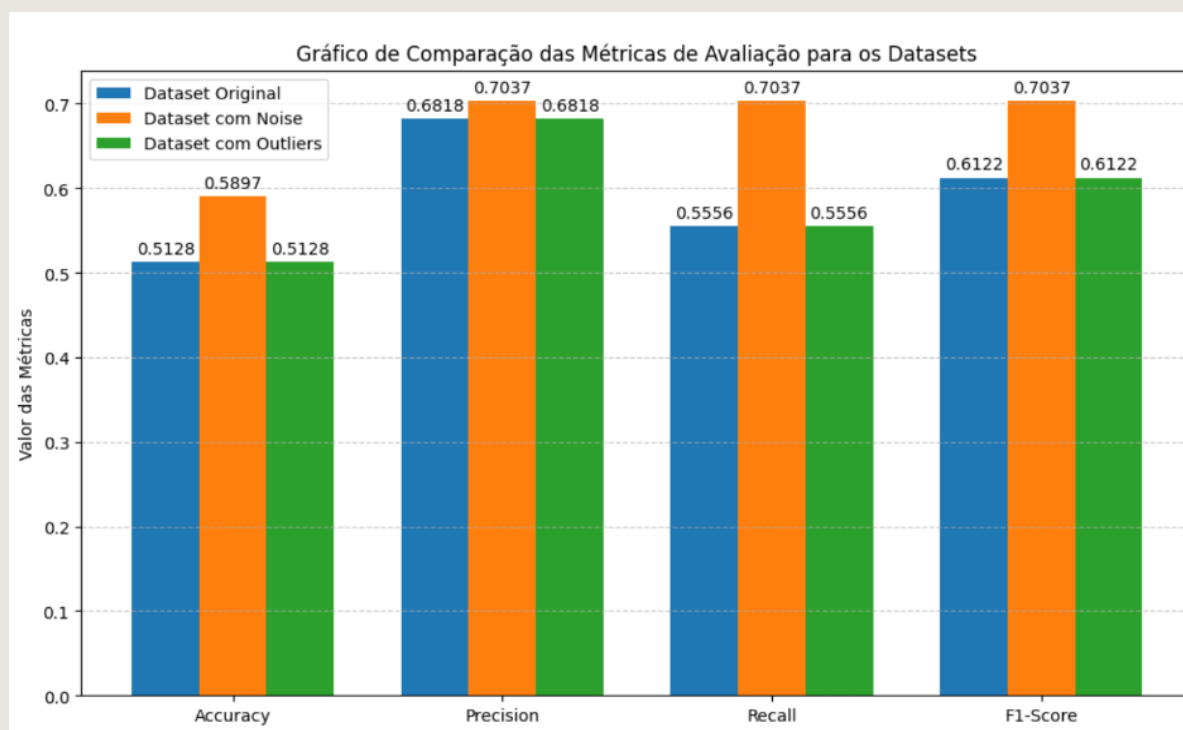
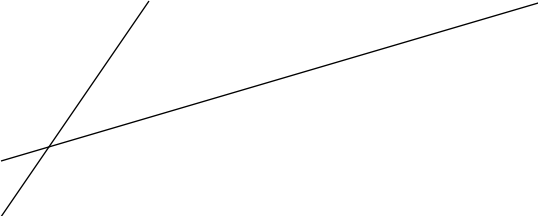


Fig.4 Gráfico de Comparação de Métricas de Avaliação para o Dataset 738_pharynx



No geral:

- O **KNN** é um **algoritmo simples** mas extremamente dependente da qualidade dos dados
- O **noise tem um impacto variável**: pequenas quantidades podem ser benéficas mas níveis mais elevados diminuem muito o desempenho
- Os **outliers têm um impacto quase sempre negativo**, perturbando os datasets e levando a classificações erradas

PROPOSAL

Motivação

Durante o estudo, verificou-se que o KNN é extremamente **sensível** à presença de **noise nas classes** e de **outliers nas características**.

Pequenas perturbações nos dados são suficientes para diminuir significativamente o desempenho do algoritmo, resultando em reduções nas métricas de avaliação.

Por isso, torna-se essencial introduzir melhorias que **aumentem a robustez do KNN** em cenários com dados ruidosos e contaminados por outliers.

PROPOSAL

Descrição

Para aumentar a robustez do KNN face a outliers e noise, propomos:

1. Identificação de vizinhos extremos com o **Modified Z-Score** (MZS)
2. Atribuição de **pesos** com base no MZS:
 - Vizinhos com valores de MZS mais elevados (mais distantes do valor padrão) recebem pesos menores

PROPOSAL

Descrição

Assim, após o cálculo das distâncias, aplicamos a:

Fórmula do Modified Z-Score:

$$MZS_i = 0,6745 \cdot \frac{X_i - Mediana}{MAD}$$

X_i : distância do vizinho ao ponto de teste

Mediana: valor central das distâncias

MAD (Median Absolute Deviation):

$$MAD = Mediana(|X_i - Mediana|)$$

Fórmula do Peso:

$$\omega_i = \frac{1}{1 + |MZS_i|} \cdot \frac{1}{X_i + \epsilon}$$

$\epsilon : 10^{-6}$ -> serve para evitar divisão por zero

ESTUDO EMPÍRICO

Começámos por aplicar o **Modified Z-Score com pesos**, sem outras alterações ao programa.

Esta versão foi testada no dataset **451_Irish**, usando os mesmos níveis de noise e outliers definidos anteriormente.

Obtivemos assim o mesmo gráfico, mas com diferentes valores.

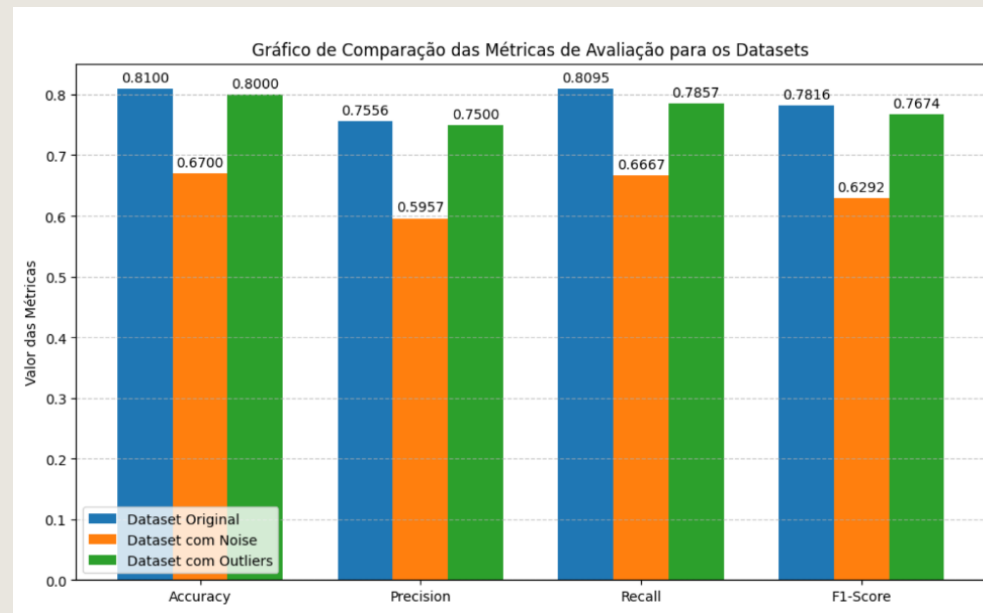
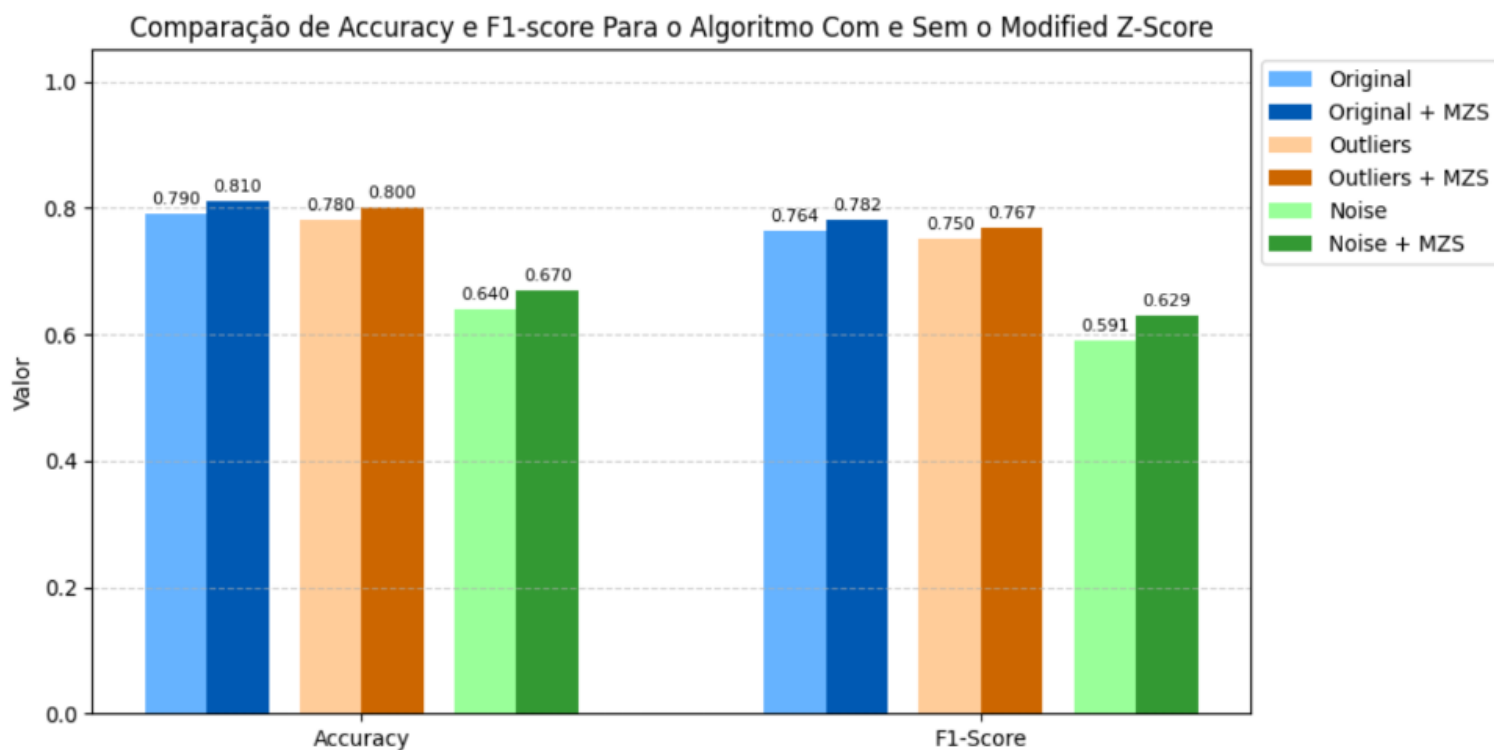


Fig. 5 Gráfico de Comparação de Métricas de Avaliação para o Dataset 451_Irish mas agora com a implementação de melhorias

Para facilitar a análise comparativa, criámos uma função que gera **gráficos de colunas** com os valores de **Accuracy** e **F1-score**, com e sem o uso do Modified Z-Score.

Optámos por estas duas métricas porque:

- **Accuracy** : mede a porporção de previsões corretas;
- **F1-score** equilibra **Precision** e **Recall**



Observa-se uma melhoria nos resultados ao aplicar o MZS, especialmente nos casos com noise e outliers.

Logo, consideramos que a introdução do MZS melhora o desempenho do KNN.

Fig. 6 Gráfico de colunas do dataset **_451_Irish** para o algoritmo com e sem as melhorias

Quanto aos outros datasets que analisamos anteriormente:

50_Tic-Tac-Toe:

Outliers ajudaram a separar melhor as classes, melhorando a performance. A aplicação do MZS **preservou os bons resultados**, mostrando-se neutra em contextos já favoráveis.

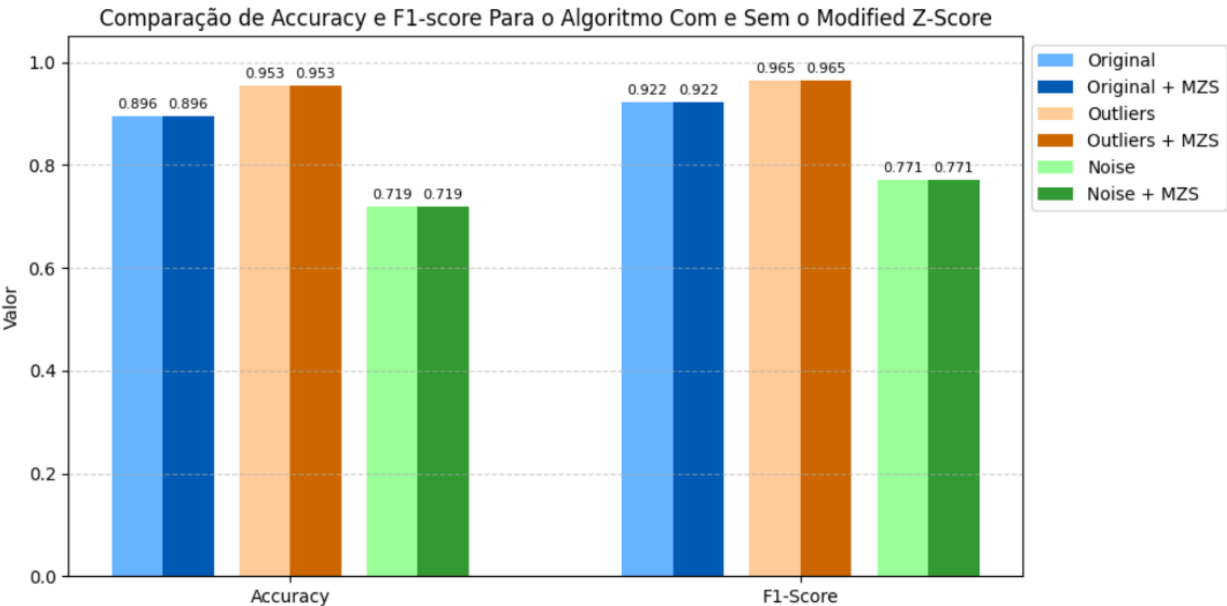


Fig. 7 Resultados obtidos no dataset 50_tic_tac_toe

738_Pharynx:

Pequenas quantidades de noise **atuaram como regularização**, melhorando o desempenho. O MZS **não trouxe melhorias adicionais** — por outro lado, reduziu ligeiramente os resultados.

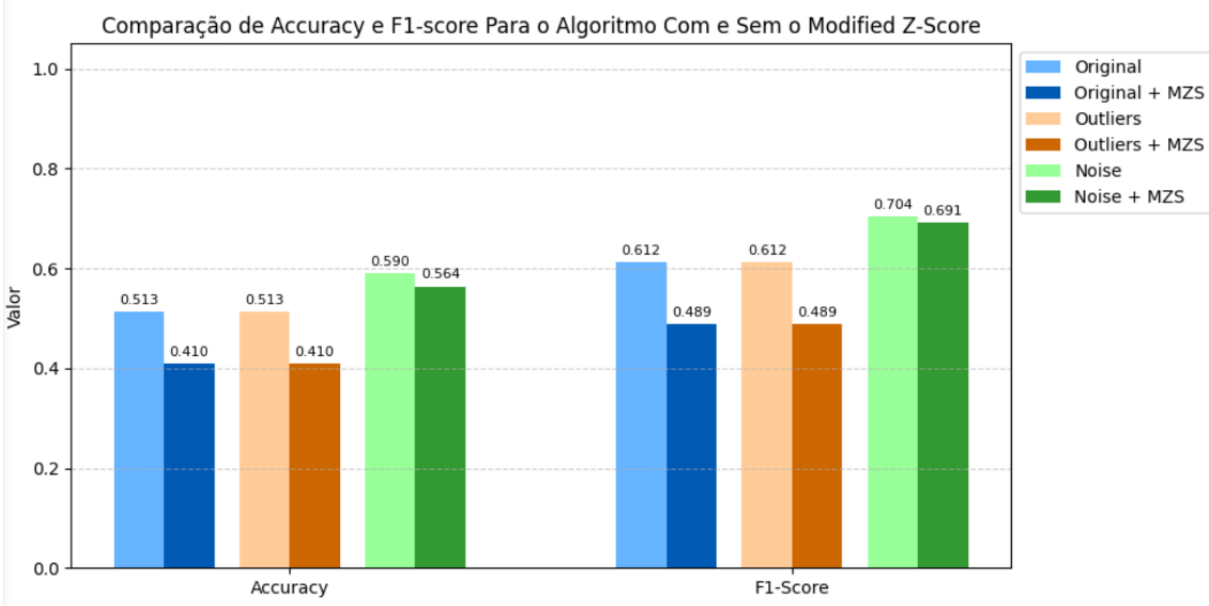


Fig. 8 Resultados obtidos no dataset 738_pharynx

Impacto de Diferentes Níveis de Outliers

→ Também decidimos analisar o comportamento do KNN com e sem MZS em cenários com diferentes **níveis de outliers**. Para isso construímos gráficos de variação da **Accuracy** e do **F1-score**.

Resultados:

- O desempenho do algoritmo **degrada-se ligeiramente** à medida que aumenta o número de outliers;
- Os resultados para o algoritmo com o MZS são sempre superiores independentemente do valor de outliers introduzidos ;

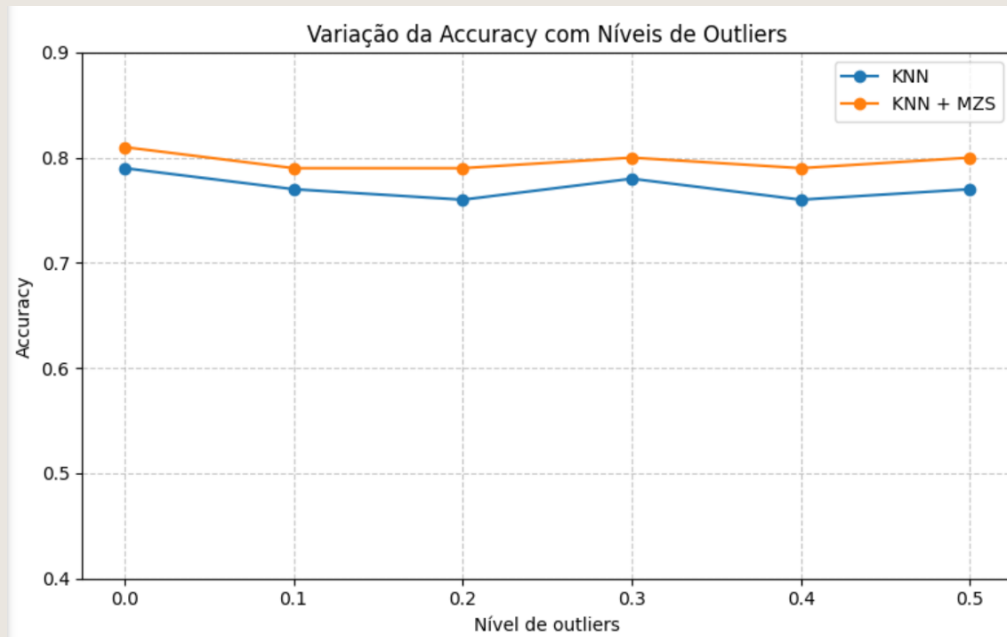


Fig. 9 Gráfico de linhas da variação da Accuracy para diferentes níveis de Outliers

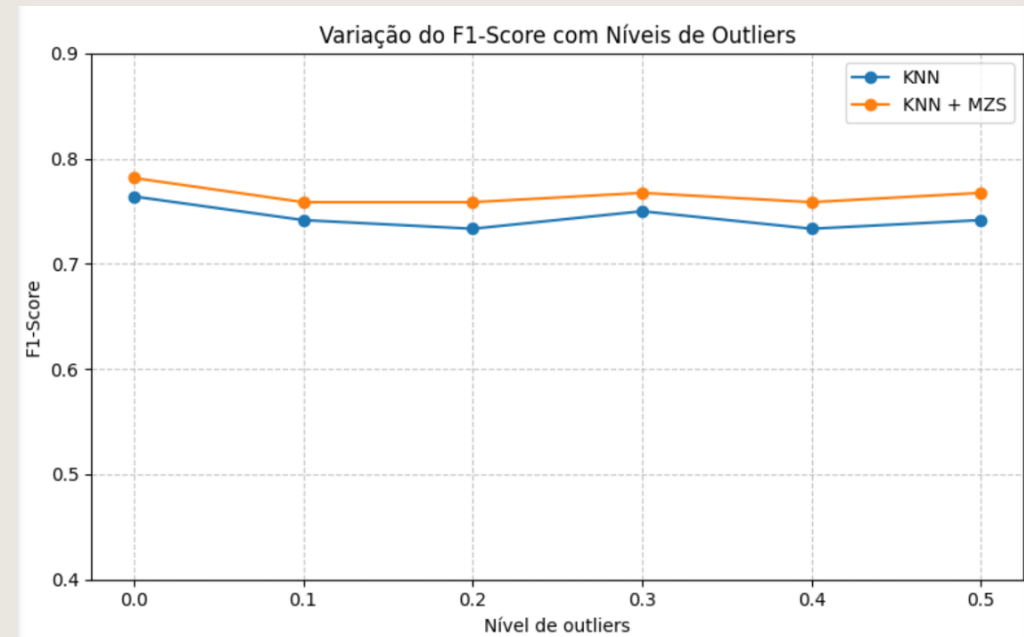


Fig. 10 Gráfico de linhas da variação do F1-Score para diferentes níveis de Outliers

Impacto de Diferentes Níveis de Noise

→ Realizamos o mesmo estudo para diferentes níveis de noise.

Resultados:

- Ao contrário dos outliers, o uso do **MZS não trouxe melhorias evidentes**.
- Tanto a **Accuracy** como o **F1-score** diminuem de forma semelhante com e sem o MZS.

Isto poderá acontecer porque:

- O **MZS não consegue filtrar os vizinhos com classe errada** quando eles estão perto. Portanto, o **noise nas classes** continua a afetar negativamente o KNN — com ou sem MZS.

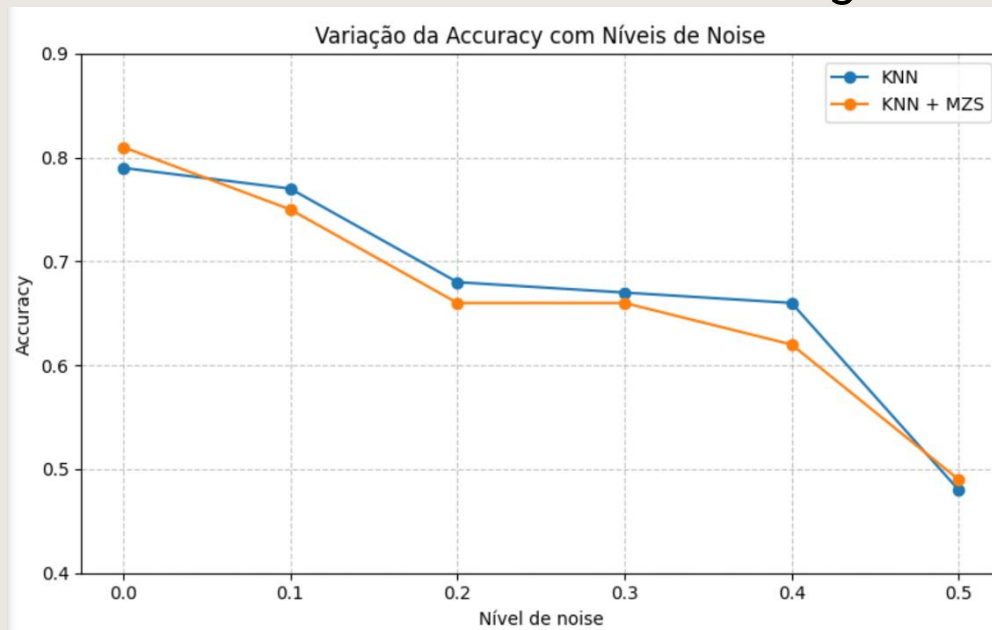


Fig. 11 Gráfico de linhas da variação da Accuracy para diferentes níveis de Noise

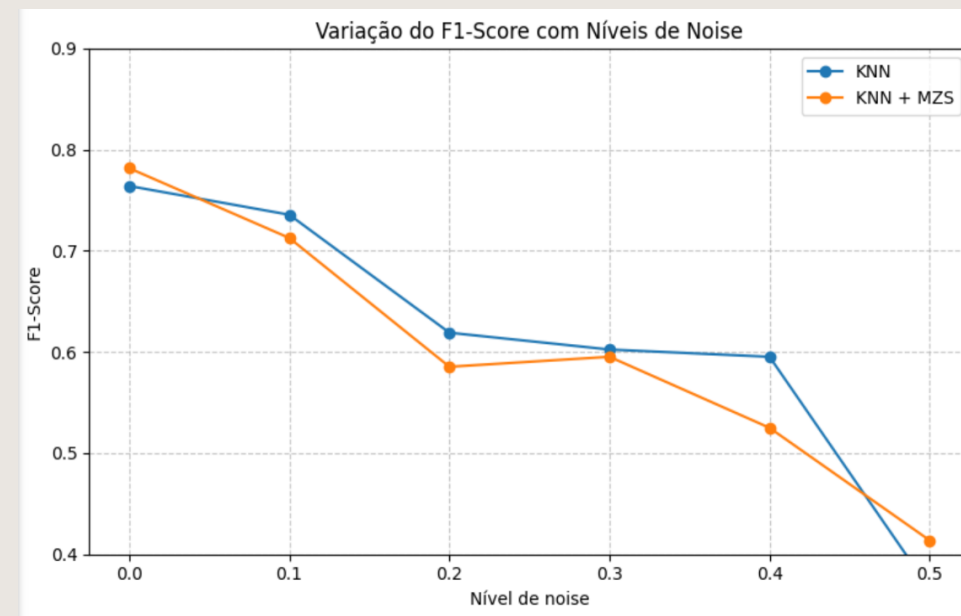
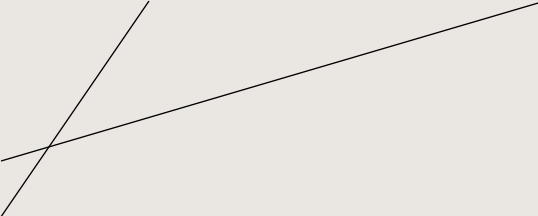


Fig. 12 Gráfico de linhas da variação do F1-Score para diferentes níveis de Noise



Ao analisarmos o impacto do nível de noise ou a quantidade de outliers introduzidos nos datasets, observámos que:

- as **métricas de avaliação tendem a diminuir** com o **aumento do noise e dos outliers**.
- mesmo adicionando um **pequeno nível de ruído** é suficiente para **provocar alterações** nas métricas.
- enquanto que, no caso dos **outliers**, é geralmente necessário introduzir uma **maior quantidade** de outliers para que se note o impacto.

Isto acontece porque o noise afeta diretamente as classes, enquanto que os outliers apenas alteram as distâncias entre os exemplos, dificultando a identificação correta dos vizinhos mais próximos, sendo necessária uma maior quantidade para afetar estes vizinhos de forma significativa.

HIPERPARÂMETRO K

O valor de k é o principal **hiperparâmetro do KNN**, definindo o número de vizinhos usados na decisão.

Para estudar o seu impacto, avaliámos a **accuracy** com $k \in \{1,3,5,7,9\}$ em diferentes cenários (original, com noise, com outliers e com MZS)

Com o gráfico podemos analisar que:

- A performance tende a melhorar até $k = 5$, e a estabilizar com um k superior
- O KNN com o MZS tem resultados melhores ou iguais ao algoritmo original, mostrando que é estável face a variações de k

→ **Conclusão:** A escolha de k influencia a performance, mas o MZS mantém-se eficaz e robusto

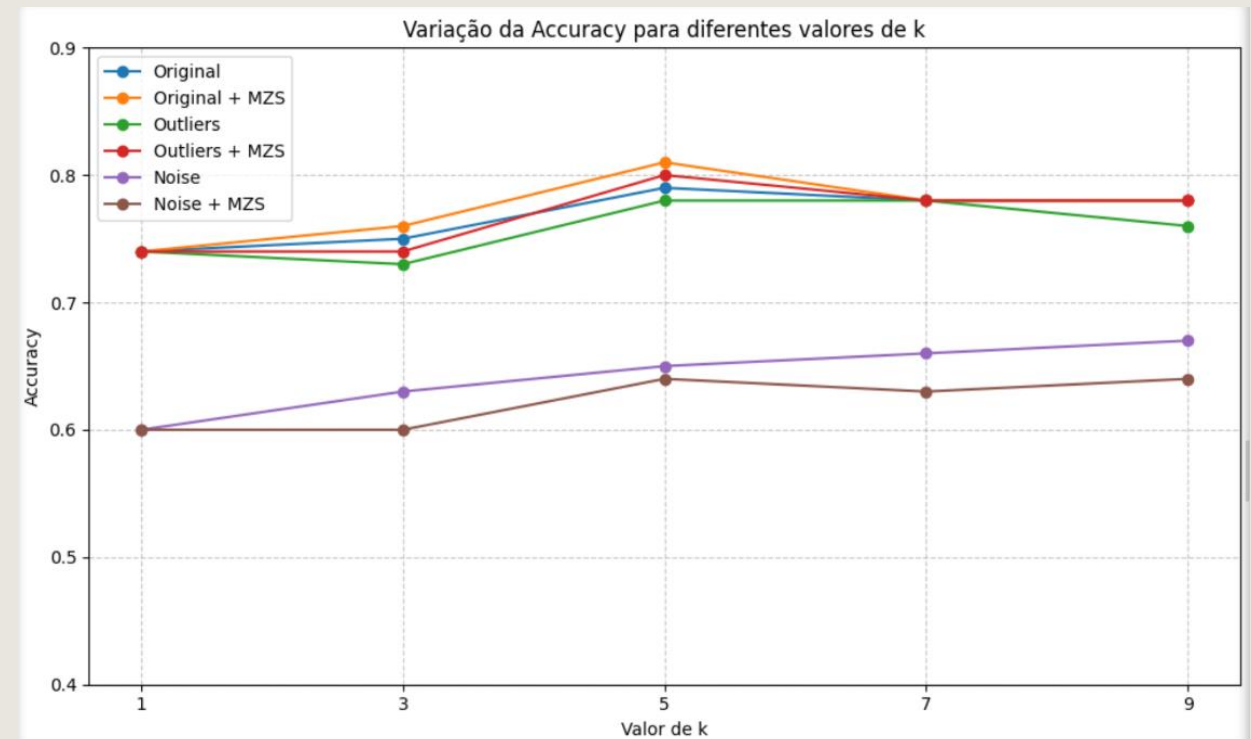


Fig. 13 Gráfico de linhas para a variação da Accuracy para diferentes valores de k

ESTIMATIVAS DE DESEMPENHO

Utilizamos **k-fold Cross Validation** (com $k = 5$) para avaliar a performance dos modelos em diferentes cenários.

Para cada teste, apresentamos a **média** e o **desvio padrão** das métricas que analisamos anteriormente.

A tabela abaixo mostra que:

- O KNN **melhora ligeiramente a performance** nos cenários do dataset **original** e nos datasets com **outliers**.
- Nos casos com **noise**, o MZS **não apresenta muitas melhorias**, o que indica que este não é suficiente para lidar com este problema.

Resultados do Cross-Validation (média ± desvio padrão)				
Cenário	Accuracy	Precision	Recall	F1-score
Original	0.734 ± 0.036	0.713 ± 0.079	0.671 ± 0.069	0.688 ± 0.058
Original + MZS	0.736 ± 0.021	0.713 ± 0.070	0.680 ± 0.032	0.694 ± 0.036
Outliers	0.730 ± 0.036	0.708 ± 0.075	0.667 ± 0.074	0.683 ± 0.058
Outliers + MZS	0.736 ± 0.022	0.715 ± 0.069	0.676 ± 0.036	0.693 ± 0.036
Noise	0.626 ± 0.024	0.567 ± 0.045	0.673 ± 0.069	0.613 ± 0.037
Noise + MZS	0.580 ± 0.038	0.525 ± 0.022	0.597 ± 0.032	0.558 ± 0.015

Fig. 14 Tabela dos resultados do k-fold Cross-Validation

ANÁLISE ESTATÍSTICA DOS RESULTADOS

- Para avaliar se as diferenças observadas são estatisticamente significativas, aplicámos os testes **t-test** e **Wilcoxon**.
- Ambos avaliam se há diferença entre duas distribuições de resultados:
 - $p < 0.05$ → diferença significativa
 - $p \geq 0.05$ → diferença não significativa
- A tabela mostra que:
 - O **noise** tem impacto estatisticamente significativo ($p < 0.05$).
 - As melhorias com o **MZS** não são estatisticamente significativas ($p > 0.05$), embora mostrem tendência positiva.

→ O KNN com MZS é **estável** e **não piora**. Apesar de haver uma melhoria nas métricas, como analisamos anteriormente, esta não é suficiente para garantir significância estatística.

	T-test	Wilcoxon
Original vs Outliers	$p = 0.3197$	$p = 0.3171$
Original vs Noise	$p = 0.0155$	$p = 0.0163$
Original vs Original + MZS	$p = 0.1583$	$p = 0.1573$
Outliers vs Outliers + MZS	$p = 0.1583$	$p = 0.1573$
Noise vs Noise + MZS	$p = 0.2589$	$p = 0.2568$



Para comparar **mais de dois métodos simultaneamente**, utilizámos o **teste de Friedman**. Este teste foi aplicado aos 6 cenários: Original, Original + MZS, Outliers, Outliers + MZS, Noise e Noise + MZS.

O resultado obtido foi: $p = 0.0001$

→ **Conclusões:**

- **O teste de Friedman** indicou diferenças estatisticamente significativas entre os 6 métodos.
- Isso mostra que o tipo de dados (original, com noise ou outliers) e o uso do MZS **influenciam o desempenho do KNN**. No entanto, os **testes anteriores** (t-test e Wilcoxon) mostraram que as melhorias trazidas pelo MZS **não foram estatisticamente significativas na maioria das comparações**.

ANÁLISE DOS RESULTADOS FACE À HIPÓTESE INICIAL

- Partimos da hipótese de que o uso do **Modified Z-Score (MZS)** melhorasse o desempenho do KNN.
- De forma geral, o MZS **aumenta a performance**, principalmente com outliers.
- No entanto, **as melhorias não foram estatisticamente significativas** nos testes par-a-par.

→ Conclusão:

- Embora a adição do Modified Z-score com pesos **não garanta melhorias estatisticamente significativas**, ele **melhora de forma consistente os resultados**, sem nunca piorar o desempenho.
- Desta forma, consideramos que o uso do MZS pode ser considerado como uma **boa prática de melhoria do KNN**, pois a sua capacidade de melhorar ou, no mínimo, manter a performance sem risco de prejudicar o modelo tornando-o uma **estratégia vantajosa**.

CONCLUSÃO E TRABALHOS FUTUROS

- O **KNN mostrou-se sensível** à presença de **noise** e **outliers**.
- O uso de pesos com **Modified Z-Score (MZS)** pode melhorar o desempenho, sobretudo em cenários com **outliers**.
- Apesar de não garantir melhorias estatisticamente significativas, o MZS oferece um **comportamento mais estável e seguro**.

Trabalhos Futuros:

- Implementar técnicas que identifiquem exemplos com noise nos datasets e ajustar a sua influência no processo de decisão do KNN, por exemplo através de ponderações.

BIBLIOGRAFIA

- Aashwin Kumar. (2022, Novembro 18). *How to handle outliers in machine learning*. DEV Community. <https://dev.to/aashwinkumar/how-to-handle-outliers-in-machine-learning-3eo2>
- GeeksforGeeks. (2021, Abril 14). *How to handle noise in machine learning?* <https://www.geeksforgeeks.org/how-to-handle-noise-in-machine-learning/>
- National Institute of Standards and Technology (NIST). (n.d.). *Outlier detection*. In *Engineering Statistics Handbook*. <https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/stable/modules/neighbors.html>
- Moreira, J. M., de Carvalho, A. C. P. L. F., & Horváth, T. (2019). *A general introduction to data analytics*.
- Gama, J., Carvalho, A. P. L., Faceli, K., Lorena, A. C., & Oliveira, M. (2017). *Extração de conhecimento de dados: Data mining* (3ª ed.). Edições Sílabo.

ANEXOS

Para além dos gráficos iniciais, o programa que desenvolvemos também retorna:

- As previsões:

```
DATASET SEM ALTERAÇÕES
Previsões: ['taken', 'not_taken', 'not_taken', 'not_taken', 'taken', 'taken', 'not_taken', 'taken', 'not_taken', 'not_taken', 'not_
taken', 'not_taken', 'not_taken', 'not_taken', 'not_taken', 'taken', 'taken', 'not_taken', 'not_taken', 'not_taken', 'not_taken', '
taken', 'taken', 'not_taken', 'not_taken', 'not_taken', 'taken', 'not_taken', 'not_taken', 'taken', 'not_taken', 'taken', 'not_take
n', 'taken', 'taken', 'not_taken', 'not_taken', 'taken', 'not_taken', 'not_taken', 'taken', 'not_taken', 'taken', 'not_taken', 'tak
en', 'taken', 'not_taken', 'not_taken', 'taken', 'taken', 'not_taken', 'taken', 'not_taken', 'not_taken', 'taken', 'not_taken', 'ta
ken', 'taken', 'taken', 'not_taken', 'not_taken', 'not_taken', 'not_taken', 'taken', 'taken', 'not_taken', 'not_taken', 'not_taken'
, 'taken', 'not_taken', 'not_taken', 'not_taken', 'not_taken', 'taken', 'taken', 'taken', 'taken', 'taken', 'taken', 'taken', 'not_
taken', 'not_taken', 'taken', 'not_taken', 'taken', 'taken', 'taken', 'taken', 'not_taken', 'taken', 'taken', 'taken', 'not_taken',
'not_taken', 'taken', 'taken', 'not_taken', 'not_taken', 'taken', 'taken']
Accuracy: 0.7900
Precision: 0.7234
Recall: 0.8095
F1-score: 0.7640
```

Fig.15 Parte do Resultado que o Programa retorna

- As Matrizes de Confusão:

Matriz de Confusão - Dataset sem Alterações

	True taken	True not_taken
Predicted taken	TP = 34	FP = 13
Predicted not_taken	FN = 8	TN = 45

Fig.16 Matriz de Confusão para o dataset sem alterações do 451_Irish